Lite框架源码理解--通过模型文件认识OPT工具

目录

1 OptBase

/lite/api/tools/opt.cc 的main函数利用googleflag工具解析完命令行参数后，调用opt.Run()进入opt的处理逻辑。

```
90  int main(int argc, char** argv) {
91    auto opt = paddle::lite_api::OptBase();
92    // If there is none input argument, print help info.
93  >   if (argc < 2) {…
```

> 以通常的转换fp16模型为例，./opt --model_file= --param_file= --enable_fp16=true --valid_target=arm

此时，opt会根据具体的命令行参数进行配置。

```
117   if (FLAGS_valid_targets != "") {      huzhiqiang
118     if (FLAGS_enable_fp16) opt.EnableFloat16();
119     opt.SetValidPlaces(FLAGS_valid_targets);
120   }
```

```
170
171     opt.Run();
172     return 0;
173   }
```

而OptBase在构造时，首先会更新当前支持的op信息。

```
44   class LITE_API OptBase {
45    public:
46     OptBase() { InitSupportedOpInfo(); }
```

```
797   void OptBase::InitSupportedOpInfo() {      weishengying, 13个月前
798     // collected targets in compile time, which are in head file
799     // supported_kernel_op_info.h
800     std::vector<std::string> collect_targets = {"kUnk",
```

supported_kernel_op_info.h在cmake build时被创建。/lite/kernels/CmakeLists.txt

```
128  # create headfile to restore ops info sorted by suppported platforms
129  execute_process(
130      COMMAND ${PYTHON_EXECUTABLE} ${PADDLE_SOURCE_DIR}/lite/tools/cmake_tools/record_supported_kernel_op.py
131      ${kernels_src_list}
132      ${fake_kernels_src_list}
133      ${ops_src_list}
134      ${PADDLE_BINARY_DIR}/supported_kernel_op_info.h      daquexian, 5个月前 via PR #8985 • [Hackathon No.65] Refactor and
135      ${LITE_BUILD_EXTRA}
136      RESULT_VARIABLE result
137      ERROR_VARIABLE error)
138  if(NOT "${error}" STREQUAL "")
139      message(FATAL_ERROR ${error})
140  endif()
```

build.opt/supported_kernel_op_info.h主要由两个数据结构组成，其中，**supported_ops_target**（vector）按targets的固定顺序记录了每个target支持的ops

```
21   const std::vector<std::vector<std::string>> supported_ops_target = {
22
23       // kUnk_OPS:
24       {"fake_quantize_range_abs_max","fake_quantize_abs_max","fake_channel_wise_quant
25
26       // kHost_OPS:
27       {"fill_constant_batch_size_like","unsqueeze","sigmoid","uniform_random","flatte
28
29       // kX86_OPS:
30       {"nearest_interp","fusion_elementwise_min_activation","search_seq_arithmetic","
```

另外一个数据结构**supported_ops**（map）记录了当前Lite支持的ops以及支持该op的targets。

```
66  const std::map<std::string, std::vector<std::string>> supported_ops={
67      {"lrn", { "kARM", "kOpenCL", "kXPU" }},
68      {"search_grnn", { "kX86", "kCUDA", "kXPU" }},
69      {"write_back", { "kHost" }},
70      {"sequence_softmax", { "kHost", "kXPU" }},
71      {"__xpu__mmdnn_bid_emb_grnn_att", { "kXPU" }},
72      {"__xpu__mmdnn_bid_emb_grnn_att2", { "kXPU" }},
73      {"__xpu__mmdnn_bid_emb_att", { "kXPU" }},
74      {"__xpu__mmdnn_match_conv_topk", { "kXPU" }},
75      {"__xpu__mmdnn_merge_all", { "kXPU" }},
76      {"search_aligned_mat_mul", { "kX86", "kCUDA" }},
```

回到OptBase::InitSupportedOpInfo()，collect_targets记录了targets的固定顺序。

```
800   std::vector<std::string> collect_targets = {"kUnk",
801                                               "kHost",
802                                               "kX86",
803                                               "kCUDA",
804                                               "kARM",
805                                               "kOpenCL",
806                                               "kAny",
807                                               "kFPGA",
```

valid_target记录了设备确切的名称。

```
817   // ignore some old targets
818   std::set<std::string> valid_target{"kARM",
819                                      "kOpenCL",
820                                      "kMetal",
821                                      "kXPU",
822                                      "kHost",
823                                      "kIntelFPGA",
824                                      "kX86",
825                                      "kBM",
826                                      "kMLU",
827                                      "huawei_ascend_npu",
828                                      "mediatek_apu",
829                                      "rockchip_npu",
830                                      "huawei_kirin_npu",
```

接着，对当前valid的target，将其支持的op信息保存在target_supported_ops_(map)中，同样的，将op对应的target信息，保存在all_supported_ops_（map）中。

```
838   for (size_t idx = 0; idx < supported_ops_target.size(); idx++) {
839     if (valid_target.find(collect_targets[idx]) != valid_target.end()) {
840       auto& support_ops = target_supported_ops_[collect_targets[idx]];
841       support_ops.insert(supported_ops_target[idx].begin(),
842                          supported_ops_target[idx].end());
843     }
844   }
845
846   for (auto& elem : supported_ops) {
847     for (auto target : elem.second) {
848       all_supported_ops_[elem.first].insert(target);
849     }
850   }
```

target_supported_ops_和all_supported_ops_是OptBase的数据成员。

```
125   bool record_strip_info_{false};
126   std::map<std::string, std::set<std::string>> target_supported_ops_{};
127   std::map<std::string, std::set<std::string>> all_supported_ops_{};
128   void RunOptimizeFromModelSet(bool record_strip_info = false);
129   void InitSupportedOpInfo();
130 };
```

至此，target支持的ops信息保存在**target_supported_ops_**中，每个op对应的target信息保存在**all_supported_ops_**中。OptBase::InitSupportedOpInfo()任务完成。

2 OptBase::Run()

```
254 void OptBase::Run() {
255   CheckIfModelSupported(false);
256   OpKernelInfoCollector::Global().SetKernel2path(kernel2path_map);
257   opt_config_.set_valid_places(valid_places_);
258   if (model_set_dir_ != "") {
259     RunOptimizeFromModelSet(record_strip_info_);
260   } else {
261     auto opt_predictor = lite_api::CreatePaddlePredictor(opt_config_);
262     opt_predictor->SaveOptimizedModel(
263         lite_out_name_, model_type_, record_strip_info_);
264   }
265 }
```

此前，opt已经经过如下配置

```
114   if (FLAGS_optimize_out != "") {
115     opt.SetOptimizeOut(FLAGS_optimize_out);
116   }
```

```
246 void OptBase::SetOptimizeOut(const std::string& lite_out_name) {
247   lite_out_name_ = lite_out_name;
248 }
```

```
119   // filename of the optimized_model
120   std::string lite_out_name_;
```

```
117   if (FLAGS_valid_targets != "") {
118     if (FLAGS_enable_fp16) opt.EnableFloat16();
119     opt.SetValidPlaces(FLAGS_valid_targets);
120   }
```

```
51   void EnableFloat16() { enable_fp16_ = true; }
```

```
96  void OptBase::SetValidPlaces(const std::string& valid_places) {
97    valid_places_.clear();
98    auto target_reprs = lite::Split(valid_places, ",");
99    std::vector<std::string> nnadapter_device_names;
100   for (auto& target_repr : target_reprs) {
101     if (target_repr == "arm") {      huzhiqiang, 3年前 via PR #3209 • [P
102       if (enable_fp16_) {
103         valid_places_.emplace_back(
104             Place{TARGET(kARM), PRECISION(kFP16), DATALAYOUT(kNCHW)});
105       }
106       valid_places_.emplace_back(
107           Place{TARGET(kARM), PRECISION(kFloat), DATALAYOUT(kNCHW)});
108       valid_places_.emplace_back(
109           Place{TARGET(kARM), PRECISION(kInt32), DATALAYOUT(kNCHW)});
110       valid_places_.emplace_back(
111           Place{TARGET(kARM), PRECISION(kInt64), DATALAYOUT(kNCHW)});
112       valid_places_.emplace_back(
113           Place{TARGET(kARM), PRECISION(kAny), DATALAYOUT(kNCHW)});
114     } else if (target_repr == "opencl") {
```

此时，valid_places_中已经保存了与KARM相关的五种Place信息。

2.1 CheckIfModelSupported

2.1.1 parse valid places and valid targets

对于当前要转换的target（--valid_target=），将该平台支持的ops和host以及unk支持的ops统计起来，存放在数据结构valid_ops中，便于后续判断该

模型是否可以顺利的run。

```
637   // 1. parse valid places and valid targets
638   auto valid_ops = target_supported_ops_.at("kHost");
639   auto valid_unktype_ops = target_supported_ops_.at("kUnk");
640   valid_ops.insert(valid_unktype_ops.begin(), valid_unktype_ops.end());
641   for (size_t i = 0; i < valid_places_.size(); i++) {
642     std::string target = TargetRepr(valid_places_[i].target);
643     // get valid ops
644     if (target == "kNNAdapter") {
645       CHECK(opt_config_.nnadapter_device_names().size());
646       for (auto& device : opt_config_.nnadapter_device_names()) {
647         auto ops = target_supported_ops_.at(device);
648         valid_ops.insert(ops.begin(), ops.end());
649       }
650     } else {
651       auto ops = target_supported_ops_.at(target);
652       valid_ops.insert(ops.begin(), ops.end());      weishengying, 13个月
653     }
654   }
```

2.1.2 Load model into program to get ops in model

```
656   // 2.Load model into program to get ops in model
657   bool is_combined_params_form = false;
658   if (!(opt_config_.model_file()).empty() &&
659       !(opt_config_.param_file()).empty()) {
660     is_combined_params_form = true;
661   }
662   std::string prog_path = lite::FindModelFileName(opt_config_.model_dir(),
663                                                   (opt_config_.model_file()),
664                                                   is_combined_params_form);
665
```

上一步已经统计了当前target可以支持的ops信息，接着统计model中需要的ops信息。

首先获得模型文件的地址，以便后续用来初始化predicitor。

模型文件可以分为combined以及非combined两种。

```
// Find correct model filename
std::string FindModelFileName(const std::string &model_dir,
                              const std::string &model_file,
                              bool combined) {
  std::string prog_path;
  if (!combined) {
    // format 1. model_dir/__model__
    // format 2. model_dir/model
    // format 3. model_dir/pdmodel
    if (IsFileExists(model_dir + "/__model__")) {
      prog_path = model_dir + "/__model__";
    } else if (IsFileExists(model_dir + "/model")) {
      prog_path = model_dir + "/model";
    } else if (IsFileExists(model_dir + "/model.pdmodel")) {
      prog_path = model_dir + "/model.pdmodel";
    } else if (IsFileExists(model_dir + "/inference.pdmodel")) {
      prog_path = model_dir + "/inference.pdmodel";
    } else {
      PrintPbModelErrorMessage();
    }
  } else {
    if (IsFileExists(model_file)) {
      prog_path = model_file;
    } else {
      LOG(FATAL) << "\nError, the model file '" << model_file
                 << "' is not existed. Please confirm that you have inputed "
                    "correct model file path.";
    }
  }
  return prog_path;
```

因此，当combined模型文件名为model.pdmodel或者inference.pdmodel时，可以使用--model_dir替代--param_file以及--model_file

```
  lite::cpp::ProgramDesc cpp_prog;      huzhiqiang, 3年前 via PR #3209 • [Pytho
  framework::proto::ProgramDesc pb_proto_prog = *lite::LoadProgram(prog_path);
  lite::pb::ProgramDesc pb_prog(&pb_proto_prog);
  // Transform to cpp::ProgramDesc
  lite::TransformProgramDescAnyToCpp(pb_prog, &cpp_prog);
```

```
std::unique_ptr<framework::proto::ProgramDesc> LoadProgram(
    const std::string &path, const lite_api::CxxModelBuffer &model_buffer) {
  std::unique_ptr<framework::proto::ProgramDesc> main_program(
      new framework::proto::ProgramDesc);
  if (model_buffer.is_empty()) {
    model_parser::BinaryFileReader file(path);
    main_program->ParseFromString(file.ReadToString(file.length()));      石晓伟
  } else {
    main_program->ParseFromString(model_buffer.get_program());
  }
  return main_program;
}
```

```
huzhiqiang, 23个月前 | 3 authors (Yan Chunwei and others)
28  class ProgramDesc : public ProgramDescAPI {
29   public:
30    ProgramDesc() = delete;
31
32    explicit ProgramDesc(framework::proto::ProgramDesc *desc) : desc_(desc) {
33      CHECK(desc_);
34    }
```

```
74    private:
75     framework::proto::ProgramDesc *desc_;  // not_own
76    };
```

接着使用读取到的framework::proto::programdesc格式数据(在framework.proto中声明，通过protobuf解析)初始化pb_prog结构。

此时，二进制格式的模型proto数据被反序列化读入Lite框架中，保存类为lite::pb（lite/model_parser/pb/program_desc.h），仅仅作为framework::proto的lite表示层。

```
huzhiqiang, 23个月前 | 3 authors (Yan Chunwei and others)
24  namespace paddle {
huzhiqiang, 23个月前 | 3 authors (Yan Chunwei and others)
25  namespace lite {
huzhiqiang, 23个月前 | 3 authors (Yan Chunwei and others)
26  namespace pb {
27        Yan Chunwei, 3年前 via PR #1800 • publ
huzhiqiang, 23个月前 | 3 authors (Yan Chunwei and others)
28  class ProgramDesc : public ProgramDescAPI {
76  };
77
78  } // namespace pb
79  } // namespace lite
80  } // namespace paddle
```

接着，遍历OpDesc，获取输入模型使用的所有op信息，并记录下当前target不支持的op信息。

```
672   std::set<std::string> unsupported_ops;
673   std::set<std::string> input_model_ops;
674   for (size_t index = 0; index < cpp_prog.BlocksSize(); index++) {
675     auto current_block = cpp_prog.GetBlock<lite::cpp::BlockDesc>(index);
676     for (size_t i = 0; i < current_block->OpsSize(); ++i) {
677       auto& op_desc = *current_block->GetOp<lite::cpp::OpDesc>(i);
678       au  std::__1::set<std::__1::string> unsupported_ops
679       in  单击显示 2 个定义。
680       if
681         unsupported_ops.insert(op_type);
682       }
683     }
684   }
```

其中cpp namespace只是一个中转层。真正的内部表示位于/lite/core/model/general/

```
#include "lite/core/model/general/block_desc.h"
#include "lite/core/model/general/op_desc.h"
#include "lite/core/model/general/program_desc.h"
#include "lite/core/model/general/var_desc.h"
津, 2个月前 | 2 authors (津 and others)
namespace paddle {        石晓伟, 2年前 via PR #3894 • cpp
津, 2个月前 | 2 authors (津 and others)
namespace lite {
津, 2个月前 | 2 authors (津 and others)
namespace cpp {
using ProgramDesc = general::ProgramDesc;
using BlockDesc = general::BlockDesc;
using OpDesc = general::OpDesc;
using VarDesc = general::VarDesc;
using OpDescWrite = general::OpDesc;
}
```

```
huzhiqiang, 14个月前 | 1 author (huzhiqiang)
namespace paddle {
huzhiqiang, 14个月前 | 1 author (huzhiqiang)
namespace lite {
huzhiqiang, 14个月前 | 1 author (huzhiqiang)
namespace general {

/*
 * The general::ProgramDesc is the internal representation for Op. All the
 * internal
 * imprementation should use it, not the pb::ProgramDesc.
 */
huzhiqiang, 14个月前 | 1 author (huzhiqiang)
class ProgramDesc : public ProgramDescAPI {
 public:
  ProgramDesc() = default;
```

2.2 SetKernel2path

和上一步类似，通过实例OpKernelInfoCollector保存每个kernel对应的file path。

```
54   void SetKernel2path(      huzhiqiang, 3年前 via PR #2256 • model
55       const std::map<std::string, std::string>& kernel2path_map) {
56     kernel2path_ = kernel2path_map;
57   }
58   const std::map<std::string, std::string>& GetOp2PathDict() {
59     return op2path_;
60   }
61   const std::map<std::string, std::string>& GetKernel2PathDict() {
62     return kernel2path_;
63   }
64
65 private:
66   std::map<std::string, std::string> op2path_;
67   std::map<std::string, std::string> kernel2path_;
68 };
```

而参数kernel2path_map来自头文件kernel_src_map.h，该文件在/lite/kernels/CMakeLists.txt生成。

```
27  // stores the map that records the source_file path of each kernel.
28  #include "kernel_src_map.h"  // NOLINT      huzhiqiang, 3年前 via PR #3209 •
```

```
20  const std::map<std::string, std::string> kernel2path_map{
21
22
23    {"slice,kARM,kFloat,kNCHW,def", "slice_compute.cc"},
24    {"slice,kARM,kFloat,kNCHW,array_def", "slice_compute.cc"},
25    {"slice,kARM,kFloat,kNCHW,float_i64_starts_ends", "slice_compute.cc"},
26    {"slice,kARM,kFloat,kNCHW,array_float_i64_starts_ends", "slice_compute.cc"},
27    {"slice,kARM,kFloat,kNCHW,bool_slice", "slice_compute.cc"},
28    {"slice,kARM,kFloat,kNCHW,array_bool_slice", "slice_compute.cc"},
29    {"slice,kARM,kFloat,kNCHW,int32_slice", "slice_compute.cc"},
30    {"slice,kARM,kFloat,kNCHW,array_int32_slice", "slice_compute.cc"},
31    {"slice,kARM,kFloat,kNCHW,def_int64", "slice_compute.cc"},
32    {"slice,kARM,kFloat,kNCHW,array_def_int64", "slice_compute.cc"},
33    {"yolo_box,kARM,kFP16,kNCHW,def", "yolo_box_compute.cc"},
```

```
#----------------------------------post_process-----------------------------------------------------------
# tricks to create headfiles for opt
file(MAKE_DIRECTORY ${PADDLE_BINARY_DIR}/all_kernel_faked_dir)
find_package(PythonInterp REQUIRED)
execute_process(
    COMMAND ${PYTHON_EXECUTABLE} ${PADDLE_SOURCE_DIR}/lite/tools/cmake_tools/create_fake_kernel_registry.py
    ${kernels_src_list}      huzhiqiang, 14个月前 via PR #6787 • [Framework] Refine add_kernels and reduce kernels
    ${fake_kernels_src_list}
    ${PADDLE_BINARY_DIR}/all_kernel_faked_dir
    ${PADDLE_BINARY_DIR}/all_kernel_faked.h
    ${PADDLE_BINARY_DIR}/all_kernel_faked.cc
    ${PADDLE_BINARY_DIR}/kernel_src_map.h
    RESULT_VARIABLE result
    ERROR_VARIABLE error)
if(NOT "${error}" STREQUAL "")
  message(FATAL_ERROR ${error})
endif()

file(GLOB all_kernel_faked_src ${PADDLE_BINARY_DIR}/all_kernel_faked_dir/*.cc)
lite_cc_library(kernels SRCS ${KERNELS_SRC} ${all_kernel_faked_src} DEPS ${lite_kernel_deps})
add_dependencies(kernels utils)
if(LITE_WITH_CUDA)
    target_link_libraries(kernels ${cuda_kernels})
endif()
```

2.3 set_valid_places

和上一步类似，使用OptBase的valid_places数组初始化opt_config即cxx_config。

```
void OptBase::Run() {
  CheckIfModelSupported(false);
  OpKernelInfoCollector::Global().SetKernel2path(kernel2path_map);
  opt_config_.set_valid_places(valid_places_);
```

2.4 opt_predictor

```
  private:
  bool enable_fp16_{false};
   CxxConfig opt_config_;      huzhiqiang, 3年
   // valid places for the optimized_model
```

```
  if (model_set_dir_ != "") {
    RunOptimizeFromModelSet(record_strip_info_);
  } else {
    auto opt_predictor = lite_api::CreatePaddlePredictor(opt_config_);
    opt_predictor->SaveOptimizedModel(      huzhiqiang, 3年前 via PR #3209
        lite_out_name_, model_type_, record_strip_info_);
  }
}
```

2.4.1 CreatePaddlePredictor

在cxxPaddleApiImp的初始化过程中，会对当前模型网络进行图优化（pass）。

```
315  template <>
316  std::shared_ptr<PaddlePredictor> CreatePaddlePredictor(
317      const CxxConfig &config) {
318    static std::mutex mutex_conf;
319    std::unique_lock<std::mutex> lck(mutex_conf);
320    auto x = std::make_shared<lite::CxxPaddleApiImpl>();
321    x->Init(config);      Yan Chunwei, 3年前 via PR #1800 • publ
322    return x;
323  }
```

```
268  class CxxPaddleApiImpl : public lite_api::PaddlePredictor {
269   public:
270    CxxPaddleApiImpl() {
271      raw_predictor_ = std::make_shared<Predictor>();
272      status_is_cloned_ = false;      huzhiqiang, 2年前 via PR …
273    }
```

```
43  void CxxPaddleApiImpl::Init(const lite_api::CxxConfig &config) {
44    config_ = config;
45    mode_ = config.power_mode();
46    threads_ = config.threads();
47  #ifdef LITE_USE_THREAD_POOL
48    int thread_num = ThreadPool::Init(threads_);
49    if (thread_num > 1) {
50      ThreadPool::AcquireThreadPool();
51    }
52  #endif
53    if (!status_is_cloned_) {
54      auto places = config.valid_places();
55      std::vector<std::string> passes = config.get_passes_internal();
```

此处的passes_interal和mir中的passes_local不一样，前者可以通过命令行参数指定，后者存储了内部所有已实现的passes的信息。

```
128     auto *sparse_detect_pass =
129         mir::PassManager::Global().LookUp<mir::SparseConvDetectPass>(
130             "sparse_conv_detect_pass");
131     CHECK(sparse_detect_pass);
132 >   if (config.sparse_model()) {…
134 >   } else {…
138     }
139
140     raw_predictor_->Build(config, places, passes);
141   } else {
142     raw_predictor_->PrepareFeedFetch();
143     CHECK(raw_predictor_) << "The Predictor can not be nullptr in Clone mode.";
144   }
```

```
300 void Predictor::Build(const lite_api::CxxConfig &config,
301                        const std::vector<Place> &valid_places,
302                        const std::vector<std::string> &passes,
303                        lite_api::LiteModelType model_type) {
304 >   if (config.is_model_from_memory()) {…
314     } else {
315       LOG(INFO) << "Load model from file.";
316       Build(config.model_dir(),
317             config.model_file(),
318             config.param_file(),
319             valid_places,
320             passes,
321             model_type,
322             config);
323     }
324 }
```

```
359 void Predictor::Build(const std::shared_ptr<cpp::ProgramDesc> &program_desc,
360                       const std::vector<Place> &valid_places,
361                       const std::vector<std::string> &passes,
362                       const lite_api::CxxConfig &config) {
363   program_desc_ = program_desc;
364   // `inner_places` is used to optimize passes
365   std::vector<Place> inner_places = valid_places;
366 >   for (auto &valid_place : valid_places) {…
370   }
371
372 >   if (IsQuantizedMode(program_desc_)) {…
383   }
384   // XPU target must make sure to insert in front of others.
385 >   if (IsQuantizedMode(program_desc_)) {…
392   }
393   Program program(program_desc_, scope_, inner_places);
394   valid_places_ = inner_places;
```

```
396     core::KernelPickFactor factor;
397     factor.ConsiderTarget();
398     factor.ConsiderPrecision();
399     factor.ConsiderDataLayout();
400
401     exec_scope_ = program.exec_scope();
402
403     program_ = RunDefaultOptimizer(      weishengying, 11个月前 via PR #77
404         std::move(program), inner_places, factor, passes, config);
405
406     if (program_desc->HasVersion())
407       program_->set_version(program_desc->Version());
408
409     PrepareFeedFetch();
410     // Verify if the ops version of current runtime program is
411     // the same with that in models.
412     CheckPaddleOpVersions(program_desc);
413
414     // Update the runtime program to program_desc only once
415     program_->SaveRuntimProgramIntoProgramDesc(program_desc_);
416   }
```

```
124   std::unique_ptr<RuntimeProgram> RunDefaultOptimizer(
125       Program&& program,
126       const std::vector<Place>& valid_places,
127       core::KernelPickFactor kernel_pick_factor,
128       const std::vector<std::string>& passes,
129       const lite_api::CxxConfig& config) {
130     Optimizer optim(valid_places, kernel_pick_factor);
131
132 >   std::vector<std::string> passes_local{…
220 > #ifdef LITE_WITH_XPU…
222 > #endif…
264 > #if !(defined(LITE_WITH_PRECISION_PROFILE))…
267   #endif
268       }};
269
270     // skip the discarded pass
271     const std::vector<std::string> discarded_passes =
272         config.get_discarded_passes();
273     for (auto& pass : discarded_passes) {
274       auto iterator = std::find(passes_local.begin(), passes_local.end(), pass);
275       if (iterator != passes_local.end()) {
276         LOG(INFO) << "discarded pass : " << pass;
277         passes_local.erase(iterator);
278       } else {
279         LOG(INFO) << "the pass : " << pass
280                   << " dont't exit or has already discarded";
281       }
282     }
```

```
284   // It's just a workaround to avoid repeated op fusion if the filter weights
285   // are shared among sub-blocks
286 > if (program.block_size() > 1) {…
298   }
299
300   // multi_stream_analysis_pass must be in the front of
301   // runtime_context_assign_pass
302   // post_quant_dynamic_pass must be in the behind of
303   // lite_quant_dequant_fuse_pass
304   const std::string msa_pass{"multi_stream_analysis_pass"};
305   const std::string msa_depend_pass{"runtime_context_assign_pass"};
306   const std::string pqd_pass{"post_quant_dynamic_pass"};
307   const std::string pqd_depend_pass{"lite_quant_dequant_fuse_pass"};
308   const std::string fp16_pass{"fp16_attribute_pass"};
309
310 > for (const std::string& pass : passes) {…
324   }
325
326 > for (auto place : valid_places) {…
333   }
334   for (auto& pass_name : passes_local) {
335     optim.AddPass(pass_name);
336   }
337
338   return optim.Run(std::move(program));
339 }
```

```
46 std::unique_ptr<RuntimeProgram> Optimizer::Run(Program&& program) {
47   auto block_size = program.block_size();
48   for (size_t block_idx = 0; block_idx < block_size; ++block_idx) {
49     std::unique_ptr<mir::SSAGraph> graph;
50     graph.reset(new mir::SSAGraph);
51     graph->Build(program, valid_places_, block_idx);
52     graph->SetValidPlaces(valid_places_);
53     graphs_.emplace_back(std::move(graph));
54   }
55   SpecifyKernelPickTactic(kernel_pick_factor_);
56   InitTargetTypeTransformPass();
57   InitControlFlowOpUnusedInputsAndOutputsEliminatePass();
58   InitControlFlowOpSharedInputsAndOutputsPlaceSyncPass();
59
60   ApplyPasses(&graphs_);      huzhiqiang, 17个月前 via PR #6154 • Parati
61
62   exec_scope_ = program.exec_scope();
63
64   return GenRuntimeProgram(&graphs_);
65 }
```

2.4.2 SaveOptimizedModel

```
301  void CxxPaddleApiImpl::SaveOptimizedModel(const std::string &model_dir,
302                                            lite_api::LiteModelType model_type,
303                                            bool record_info) {
304    raw_predictor_->SaveModel(model_dir, model_type, record_info);
305  }
```

```
80  void Predictor::SaveModel(const std::string &dir,
81                            lite_api::LiteModelType model_type,
82                            bool record_info) {
83    if (!program_) {
84      GenRuntimeProgram();
85    }
86    switch (model_type) {
87      case lite_api::LiteModelType::kProtobuf:
88        SaveModelPb(dir, *program_->exec_scope(), *program_desc_.get(), true);
89        break;
90      case lite_api::LiteModelType::kNaiveBuffer:
91        SaveModelNaive(dir, *program_->exec_scope(), *program_desc_.get());
92        break;
93      default:
94        LOG(FATAL) << "Unknown model type";
95    }
96    if (record_info) {
97      MkDirRecur(dir);
98      SaveOpKernelInfo(dir);
99    }
100 }
```